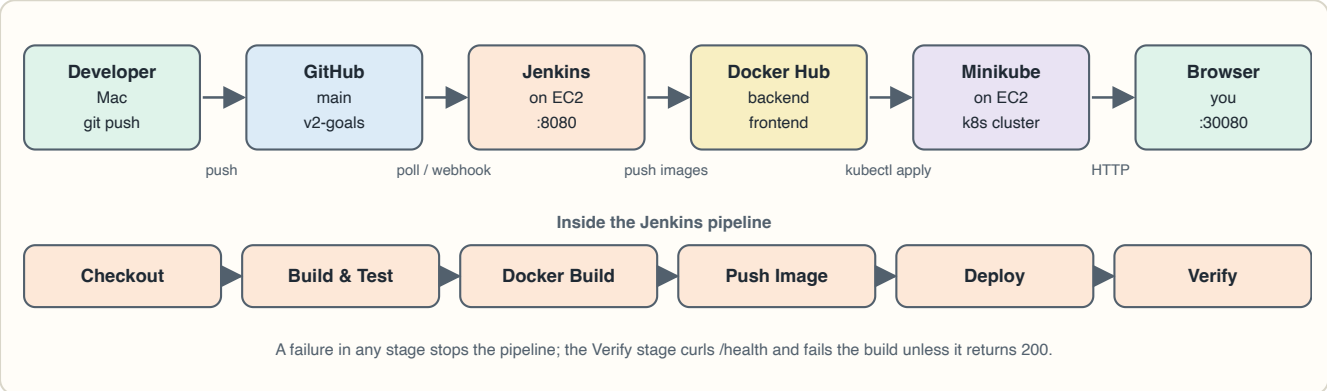
CI/CD for your own project

Jenkins + Docker + Docker Hub + Kubernetes (Minikube) on AWS. A step-by-step guide with copy-paste commands, written from a real working setup, including the problems we hit. Works for any web app that can run in a Docker container.

0. What you will build

Push code to GitHub. A Jenkins server on AWS tests it, builds a Docker image, pushes the image to Docker Hub, deploys it to a Kubernetes cluster (Minikube) on the same server, and checks that the app answers. After the one-time setup, every push is deployed automatically.



Time needed: about 3 hours the first time. **Cost:** a few dollars of AWS credit if you stop the server when not in use (section 13).

Words you will see

Term	Meaning
Image	a packaged, read-only copy of your app and everything it needs
Registry	a store for images (Docker Hub)
Pipeline	the automated steps Jenkins runs, written in a file called `Jenkinsfile`
Pod, Deployment, Service	Kubernetes objects: a running container, a manager that keeps N copies running, and a stable network address
NodePort	a Service type that opens a fixed port on the server

1. Before you start

You need these accounts and tools. Everything is free to create.

Need	Where
GitHub account and the `gh` CLI	github.com and `brew install gh` then `gh auth login`
Docker Hub account	hub.docker.com
AWS account with credits	aws.amazon.com
Docker Desktop on your laptop	docker.com (to test your image locally)
A project with a way to run tests	any language

Fill in these values once

Replace the placeholders everywhere you see them. Write your values here so you do not mix them up.

Placeholder	Your value	Example
`APP`	short lowercase app name	`myapp`
`HUBUSER`	Docker Hub username	`janedoe`
`GHUSER`	GitHub username	`janedoe`
`REPO`	GitHub repository name	`myapp-cicd`
`PORT`	port your app listens on inside the container	`5000`
`EC2IP`	public IP of your AWS server (set in section 6)	`13.1.2.3`

2. Prepare your project

Your repository root should look like this:

```
.
├── Dockerfile
├── .dockerignore
├── Jenkinsfile
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── (your code and tests)
```

2.1 Your app needs a health endpoint

Add a route that returns HTTP 200, for example `GET /health` returning `{"status":"ok"}`. Kubernetes probes and the pipeline's final check both use it. Pick the path and use the same one everywhere below.

2.2 Dockerfile

This example is for Python. Change the base image, install and start commands for your stack (see section 15).

```
FROM python:3.12-slim

ARG APP_VERSION=dev
ARG GIT_COMMIT=dev
ARG BUILD_NUMBER=dev
ENV APP_VERSION=${APP_VERSION} GIT_COMMIT=${GIT_COMMIT} BUILD_NUMBER=${BUILD_NUMBER}

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# Run as a non-root user
RUN useradd --create-home --uid 10001 appuser
USER appuser

EXPOSE PORT
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:PORT/health', timeout=2).status == 200 else 1)"

CMD ["gunicorn", "--bind", "0.0.0.0:PORT", "app:app"]
```

Why each part: copying the requirements file first lets Docker reuse the cached install layer; the build arguments become environment variables so the running app can report which build it is; running as a non-root user limits damage if the app is compromised.

2.3 .dockerignore

```
.git
.venv
node_modules
__pycache__
*.pyc
.pytest_cache
.env
k8s
Jenkinsfile
README.md
```

2.4 Test the image on your laptop first

```
docker build -t APP:local .
docker run --rm -p 8080:PORT APP:local
# in another terminal:
curl -s localhost:8080/health
```

If this fails, fix it now. It is much faster to debug here than inside Jenkins.

3. Kubernetes manifests

Put these two files in `k8s/`. The image line says `:latest`; the pipeline will rewrite it to the build number at deploy time.

k8s/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: APP
  labels:
    app: APP
spec:
  replicas: 2
  selector:
    matchLabels:
      app: APP
  template:
    metadata:
      labels:
        app: APP
    spec:
      containers:
        - name: APP
          image: HUBUSER/APP:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: PORT
          env:
            - name: HOSTNAME            # pod name via the downward API
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
          readinessProbe:
            httpGet: { path: /health, port: PORT }
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: PORT }
            initialDelaySeconds: 10
            periodSeconds: 10
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits: { cpu: 250m, memory: 256Mi }
```

k8s/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: APP
spec:
  type: NodePort
  selector:
    app: APP
  ports:
    - port: 80
      targetPort: PORT
      nodePort: 30080
```

Important: if your app keeps data in a file inside the container (for example SQLite), use `replicas: 1`. Two copies would each have their own separate data. A shared database is needed to scale.

4. The Jenkinsfile

Save this as `Jenkinsfile` in the repository root.

```
pipeline {
    agent any

    environment {
        DOCKERHUB_USER = 'HUBUSER'
        IMAGE          = "${DOCKERHUB_USER}/APP"
        APP_VERSION    = '1.0.0'
    }

    triggers {
        // Fallback when no webhook can reach Jenkins: check GitHub every 5 minutes
        pollSCM('H/5 * * * *')
    }

    options {
        timestamps()
        disableConcurrentBuilds()
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                script {
                    env.GIT_SHORT = sh(script: 'git rev-parse --short HEAD', returnStdout:
true).trim()
                }
            }
        }

        stage('Build & Test') {
            steps {
                sh '''
                    python3 -m venv .venv
                    . .venv/bin/activate
                    pip install --quiet -r requirements.txt
                    pytest -q
                '''
            }
        }

        stage('Docker Build') {
            steps {
                sh '''
                    docker build \
                        --build-arg APP_VERSION=${APP_VERSION} \
                        --build-arg GIT_COMMIT=${GIT_SHORT} \
                        --build-arg BUILD_NUMBER=${BUILD_NUMBER} \
                        -t ${IMAGE}:${BUILD_NUMBER} -t ${IMAGE}:latest .
                '''
            }
        }

        stage('Push Image') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub-creds',
                                                  usernameVariable: 'DH_USER',
                                                  passwordVariable: 'DH_TOKEN')]) {
                    // Single quotes: Groovy must not interpolate the secret into the log
                    sh '''
                        echo "$DH_TOKEN" | docker login -u "$DH_USER" --password-stdin
                        docker push ${IMAGE}:${BUILD_NUMBER}
                        docker push ${IMAGE}:latest
                    '''
                }
            }
```

```
        }

        stage('Deploy to Minikube') {
            steps {
                sh '''
                    sed -i "s|image: .*|image: ${IMAGE}:${BUILD_NUMBER}|" k8s/deployment.yaml
                    kubectl apply -f k8s/
                    kubectl rollout status deployment/APP --timeout=120s
                '''
            }
        }

        stage('Verify') {
            steps {
                sh '''
                    kubectl get pods,deploy,svc
                    MINIKUBE_IP=$(minikube ip)
                    for i in 1 2 3 4 5 6; do
                        CODE=$(curl -s -o /tmp/health.json -w "%{http_code}"
http://${MINIKUBE_IP}:30080/health || true)
                        [ "$CODE" = "200" ] && break
                        echo "Attempt $i: got $CODE, retrying..."; sleep 5
                    done
                    cat /tmp/health.json; echo
                    [ "$CODE" = "200" ] || { echo "Health check failed"; exit 1; }
                '''
            }
        }
    }

    post {
        always  { sh 'docker logout || true' }
        failure {
            sh '''
                kubectl describe deployment/APP || true
                kubectl logs deploy/APP --tail=50 || true
            '''
        }
    }
}
```

Replace `APP` and `HUBUSER` with your values. The pipeline logic, stage by stage:

Stage	What it does	Fails when
Checkout	gets the code, records the short commit id	repo unreachable
Build & Test	installs dependencies and runs your tests	any test fails (nothing is built or deployed)
Docker Build	builds the image tagged with the build number and `latest`	Dockerfile error
Push Image	logs in to Docker Hub with the stored token and pushes	wrong token or permissions
Deploy to Minikube	rewrites the image tag, applies manifests, waits for the rollout	pods never become ready
Verify	calls `/health` and requires HTTP 200	app is unhealthy

5. Put the code on GitHub

```
cd your-project
git init -b main
git add -A
git commit -m "Add Dockerfile, Jenkinsfile and Kubernetes manifests"
gh repo create REPO --public --source=. --remote=origin --push
```

The repository must be public so that Jenkins can clone it without credentials. For a private repository you need to add GitHub credentials to Jenkins as well.

6. AWS: launch the server

6.1 Protect your credits first (2 minutes)

In the AWS Console go to **Billing and Cost Management, Budgets, Create budget**, choose **Zero spend budget**, and enter your email.

6.2 Launch the instance

Go to **EC2, Launch instance** and set:

Field	Value
Name	`cicd-server`
AMI	**Ubuntu Server 24.04 LTS** (x86). Avoid the newest release (26.04) because third-party apt repositories may not support it yet. Avoid images with "Pro", "SQL Server" or "Deep Learning" in the name
Instance type	at least **2 vCPU and 4 GB RAM**. Jenkins plus Minikube need about 4 GB. `t3.medium` is ideal if your account allows it
Key pair	create new, **RSA**, **.pem**, name it `cicd-key`, download it
Storage	**30 GiB** gp3

If your account is on the AWS Free Plan, `t3.medium` is blocked ("not eligible under the Free Plan"). Do **not** upgrade the account to get it. In the instance type box look for a type with the **Free tier eligible** tag and at least 4 GB of memory. On our account `m7i-flex.large` (2 vCPU, 8 GiB) was eligible. Never use a 1 GB type (`t3.micro`): Jenkins and Minikube will freeze it.

6.3 Security group (firewall)

Under **Network settings, Create security group**, add these inbound rules, all with Source **My IP**:

Type	Port	Purpose
SSH	22	you log in
Custom TCP	8080	Jenkins web page
Custom TCP	30080	your app

Click **Launch instance**, wait for **Running** and **2/2 checks passed**, then copy the **Public IPv4 address**. This is `EC2IP`.

6.4 Connect

```
chmod 400 ~/Downloads/cicd-key.pem
ssh -i ~/Downloads/cicd-key.pem -o ServerAliveInterval=30 ubuntu@EC2IP
```

Type `yes` for the fingerprint prompt. If it hangs, your IP changed: edit the security group rules and set the Source to **My IP** again.

Check the machine:

```
lsb_release -d && nproc && free -h | head -2 && df -h / | tail -1
```

7. Install the tools on the server

Run these blocks on the server, one at a time, and stop if one fails.

7.1 Docker, Java 21, git, Python venv

Jenkins now requires **Java 21 or newer**. Java 17 will make it refuse to start.

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git gnupg python3-venv docker.io openjdk-21-jre
sudo systemctl enable --now docker
java -version 2>&1 | head -1 && docker --version && git --version
```

7.2 Jenkins

```
sudo mkdir -p /etc/apt/keyrings
sudo curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key -o
/etc/apt/keyrings/jenkins-keyring.asc
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-
stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list
sudo apt-get update && sudo apt-get install -y jenkins
sudo systemctl enable --now jenkins
sleep 30; systemctl is-active jenkins
```

If `apt-get update` shows `NO_PUBKEY` or "not signed", Jenkins has rotated its signing key again. Open https://www.jenkins.io/doc/book/installing/linux/ and use the key file name shown there in the `curl` line (the year in the file name changes).

If Jenkins says `failed`, read the cause:

```
sudo journalctl -u jenkins --no-pager | grep -v '^$' | grep -iE "java|error|exception" | head
```

7.3 kubectl and Minikube

```
curl -fsSLO "https://dl.k8s.io/release/$(curl -fsSL
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl && rm kubectl
curl -fsSLO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install -m 0755 minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64
kubectl version --client | head -1 && minikube version | head -1
```

7.4 Let Jenkins use Docker, and start Minikube as the jenkins user

This is the most important setup detail. The pipeline runs as the `jenkins` user, so Docker access and the Kubernetes cluster must belong to that user.

```
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
sudo systemctl restart jenkins
sudo -u jenkins -H minikube start --driver=docker --cpus=2 --memory=4096
```

The first start takes 2 to 4 minutes.

7.5 Verify

```
sudo -u jenkins -H minikube status
sudo -u jenkins -H kubectl get nodes
sudo -u jenkins -H minikube ip
```

You want `host`, `kubelet` and `apiserver` all Running, a node with status `Ready`, and an IP such as `192.168.49.2`.

8. Jenkins setup

8.1 First login

1. On the server print the unlock password (keep it private):

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

2. Open `http://EC2IP:8080` in your browser and paste the password into **Unlock Jenkins**.
3. Choose **Install suggested plugins** (2 to 5 minutes).
4. Create an admin user with a new username and password. Jenkins accounts are local to this server; an account from another Jenkins or from jenkins.io does not work here.
5. Keep the Jenkins URL and click **Save and Finish**.

8.2 Extra plugins

Manage Jenkins, Plugins, Available plugins. Tick **Docker Pipeline** and **Pipeline Stage View**, click **Install**, and tick **Restart Jenkins when installation is complete**. Log in again after the restart.

8.3 Docker Hub credential

Create an access token (not your password): on hub.docker.com go to **Account settings, Personal access tokens, Generate new token**, description `jenkins`, permission **Read & Write**, then copy it once. Never paste it into chat, a screenshot or a file in the repository.

In Jenkins: **Manage Jenkins, Credentials, System, Global credentials, Add Credentials**:

Field	Value
Kind	Username with password
Username	your Docker Hub username
Password	the access token
ID	`dockerhub-creds` (exactly; the Jenkinsfile uses this name)

A token is safer than a password: it can be revoked, limited to what it needs, and your real password is never stored.

8.4 Create the pipeline job

New Item, name `APP-pipeline`, type **Pipeline**, OK. In the **Pipeline** section set:

Field	Value
Definition	Pipeline script from SCM
SCM	Git
Repository URL	`https://github.com/GHUSER/REPO.git`
Branch Specifier	`*/main`
Script Path	`Jenkinsfile`

Click **Save**.

9. First run

Click **Build Now**, open the build number, then **Console Output**. The first run takes 3 to 6 minutes because images are downloaded and pushed for the first time.

Stage	Healthy output
Checkout	commit hash printed
Build & Test	your test summary, for example `65 passed`
Docker Build	`naming to docker.io/HUBUSER/APP:1`
Push Image	`Login Succeeded` then digests
Deploy to Minikube	`deployment.apps/APP created`, then `successfully rolled out`
Verify	pod list, then your health JSON, then `Finished: SUCCESS`

Docker Hub repositories created by the first push are public by default. If your image pull fails with `ImagePullBackOff`, open the repository on Docker Hub, Settings, and set it to Public.

10. Open the app in your browser

Minikube with the docker driver runs the cluster inside a private Docker network on the server (`192.168.49.2`). Port 30080 exists only there, not on the server's public IP, so `EC2IP:30080` does not work yet even though the security group allows it. The pipeline's Verify stage works because it runs on the server itself.

Fix: forward the service on all interfaces of the server.

```
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/APP 30080:80 >
/tmp/portforward.log 2>&1 &
sleep 3; cat /tmp/portforward.log; curl -s localhost:30080/health
```

Open `http://EC2IP:30080` (plain `http`). Chrome showing "Not Secure" is expected.

Two things to remember: - A port-forward is attached to one specific pod. **After every deploy, restart it:**

```
sudo pkill -f "kubectl port-forward"
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/APP 30080:80 >
/tmp/portforward.log 2>&1 &
```

- It also stops when the server restarts. Production setups use an Ingress or a load balancer instead.

Letting a friend open it

Other people's devices time out (`ERR_CONNECTION_TIMED_OUT`) because the security group only allows your IP. In EC2, open your security group, **Edit inbound rules**, and for port **30080 only** add your friend's public IP followed by `/32`, or `0.0.0.0/0` for a temporary public demo. Never open 22 or 8080 to everyone. Set it back to **My IP** afterwards.

11. Make a change and watch it deploy

On your laptop:

```
cd your-project
# edit something visible, for example a page title
git add -A && git commit -m "Change page title"
git push origin main
```

In Jenkins click **Build Now** (or wait up to 5 minutes for the poll). Watch the rollout on the server:

```
sudo -u jenkins -H kubectl get pods -w
```

New pods start, pass their readiness probe, and the old ones terminate: a rolling update with no downtime. Restart the port-forward (section 10), then hard-refresh the browser with Cmd+Shift+R.

To see the build number in the page, show the environment variables `BUILD_NUMBER`, `GIT_COMMIT` and `HOSTNAME` somewhere, for example in a footer. It makes the demo convincing.

12. Troubleshooting

Symptom	Cause and fix
SSH hangs	your IP changed: set the SSH rule's source to **My IP**
Jenkins page times out	same: the 8080 rule; or Jenkins is not active: `systemctl status jenkins`
Jenkins `NO_PUBKEY` at install	signing key rotated: see 7.2
Jenkins service failed	Java too old: install Java 21, `sudo update-alternatives --set java /usr/lib/jvm/java-21-openjdk-amd64/bin/java`, restart Jenkins
`permission denied ... /var/run/docker.sock` in a build	`sudo usermod -aG docker jenkins && sudo systemctl restart jenkins`
`minikube: host does not exist` or `connection refused` in a build	cluster not running for the jenkins user: `sudo -u jenkins -H minikube start --driver=docker --cpus=2 --memory=4096`
`ensurepip is not available`	`sudo apt-get install -y python3-venv`
`denied: requested access to the resource is denied` on push	token lacks Read & Write, or `DOCKERHUB_USER` does not match the token's account
`ImagePullBackOff`	the Docker Hub repository is private: make it Public; or the image name in the manifest has a typo
Pods `CrashLoopBackOff`	`sudo -u jenkins -H kubectl logs deploy/APP`; often a missing dependency or wrong `PORT`
Pods Running but not Ready	the health path or port in the probes is wrong
Verify stage gets 000 or 502	pods not ready yet, or wrong health path; check `kubectl get pods` and logs

Symptom	Cause and fix
Site does not load after a deploy	the port-forward died: restart it (section 10)
Site loads for you but not a friend	security group source: section 10
Data differs between page refreshes	more than one replica with file-based storage: use 1 replica or a shared database
After stopping and starting the server	IP changed, Minikube is stopped (`minikube start` as jenkins), port-forward is gone
Pipeline did not start on push	no webhook: wait for the 5-minute poll, or click Build Now
`git` commands fail on the server	your repository lives on your laptop; the server only needs Jenkins's own clone

13. Cost control and cleanup

- Set the zero-spend budget (6.1) before anything else.
- **Stop** the instance when you are not working: EC2, Instances, Instance state, **Stop**. A stopped instance does not charge for compute but still stores its disk. After a restart, use the new public IP and run `sudo -u jenkins -H minikube start --driver=docker --cpus=2 --memory=4096`.
- When the project is submitted: **Terminate** the instance, then check **EC2, Volumes** and **Elastic IPs** for anything left over and delete it.
- Revoke the Docker Hub token (Account settings, Personal access tokens, Delete) and delete unused images and repositories.

14. Command cheat sheet

On your laptop

```
docker build -t APP:local . && docker run --rm -p 8080:PORT APP:local
git add -A && git commit -m "msg" && git push origin main
ssh -i ~/Downloads/cicd-key.pem -o ServerAliveInterval=30 ubuntu@EC2IP
```

On the server

```
sudo -u jenkins -H minikube status
sudo -u jenkins -H minikube start --driver=docker --cpus=2 --memory=4096
sudo -u jenkins -H kubectl get pods,deploy,svc
sudo -u jenkins -H kubectl get pods -w
sudo -u jenkins -H kubectl logs deploy/APP --tail=50
sudo -u jenkins -H kubectl describe deploy APP
sudo -u jenkins -H kubectl rollout history deployment/APP
sudo -u jenkins -H kubectl rollout undo deployment/APP
sudo systemctl status jenkins
sudo journalctl -u jenkins --no-pager | tail -30
sudo pkill -f "kubectl port-forward"
sudo -u jenkins -H nohup kubectl port-forward --address 0.0.0.0 svc/APP 30080:80 >
/tmp/portforward.log 2>&1 &
curl -s localhost:30080/health
```

Final checklist

Done	Item
☐	App has a health endpoint and tests that pass locally
☐	Image builds and runs on your laptop
☐	`k8s/` manifests and `Jenkinsfile` have your `APP` and `HUBUSER` filled in
☐	Code pushed to a public GitHub repository
☐	EC2 running (2 vCPU, 4 GB or more), security group rules set to My IP
☐	Java 21, Docker, Jenkins, kubectl, Minikube installed
☐	Minikube started **as the jenkins user** and `Ready`
☐	Jenkins plugins installed and credential `dockerhub-creds` created
☐	Pipeline job created and first build green
☐	Port-forward running and the app opens in the browser
☐	A second change deployed automatically
☐	Budget alert set, and the instance stopped when finished

15. Adapting the pipeline to other stacks

Only three places change: the **Dockerfile**, the **Build & Test** stage, and the **health path**.

Stack	Dockerfile idea	Build & Test stage	Extra on the server
Python (Flask, Django)	`python:3.12-slim`, `pip install`, gunicorn	venv, `pip install`, `pytest`	`python3-venv`
Node.js (Express)	`node:22-alpine`, `npm ci --omit=dev`, `node server.js`	`npm ci && npm test`	`sudo apt-get install -y nodejs npm`
Java (Spring Boot)	multi-stage: Maven build, then `eclipse-temurin:21-jre` running the jar	`./mvnw -q test`	Java 21 (already installed)
Static site	`nginx:alpine` copying your files into `/usr/share/nginx/html`	none, or a lint step	nothing
React, Vue (built SPA)	multi-stage: `node` builds, `nginx:alpine` serves `dist`	`npm ci && npm test && npm run build`	Node on the server

Two images (frontend and backend). Duplicate the Docker Build, Push and Deploy steps with two image names (for example `HUBUSER/APP-backend` and `HUBUSER/APP-frontend`), make one manifest file per tier, expose only the frontend with a NodePort, and give the backend a ClusterIP Service whose name the frontend uses as its hostname (for example nginx proxying to `http://backend:5000`).

Need a pull secret? If you keep the Docker Hub repository private, create a secret and reference it:

```
sudo -u jenkins -H kubectl create secret docker-registry regcred \
  --docker-username=HUBUSER --docker-password='YOUR_TOKEN' --docker-email=you@example.com
```

Then add `imagePullSecrets: [{ name: regcred }]` under `spec.template.spec` in the Deployment. Enter the token only on the server terminal, never in chat or files.